Text-to-SQL AI Agent

Local setup and run log: Windows, VS Code, Python 3.12, Google Gemini (free tier)

1. What this project does

A natural-language-to-SQL agent built with **LangGraph**, **FastAPI** and **Gradio**. You type a business question in plain English; the agent picks the relevant tables, writes a SQLite SELECT query, blocks anything unsafe, runs it, checks the result for plausibility, and returns a plain-English summary. The original project used Claude Haiku; this run swaps it for Google Gemini's free tier.

Agent pipeline

```
question
  -> cache_check     (repeat question? reuse the stored SQL)
  -> schema_filter   (pick relevant tables; translate cryptic column names)
  -> disambiguate    (state an assumption for vague words like 'revenue')
  -> generate_sql    (LLM writes the SELECT query)
  -> sql_guard       (block DROP / DELETE / UPDATE / multi-statement SQL)
  -> execute_sql     (retry up to 2x with the error message)
  -> validate_result (LLM checks the numbers look plausible)
  -> summarize       (plain-English answer)
  -> cache_store
```

Files

File	Role
agent.py	LangGraph graph with all 9 nodes; the only file changed for Gemini
api.py	FastAPI backend on port 8004
gradio_ui.py	Browser UI on port 7860 (Chat, DB Browser, Schema, Cache, Cost, Evaluate, SQL Guard)
schema_index.py	Table relevance scoring and abbreviation map
sql_guard.py	Deterministic SELECT-only enforcement
query_cache.py / token_tracker.py	Query cache and token-usage log (SQLite)
seed_db.py / ugly_schema_seed.py	Create business.db (4 tables) and enterprise.db (25+ tables)
eval_harness.py	10-case benchmark with silent-wrong-answer detection

2. Environment

Item	Value
OS / editor	Windows, VS Code, PowerShell terminal
Python	3.12 (installed with winget). Python 3.15 failed, see section 4
LLM	Google Gemini via langchain-google-genai, model gemini-flash-lite-latest
API key	Free key from Google AI Studio, stored in .env as GOOGLE_API_KEY

3. Setup steps

Run in the VS Code terminal from the project folder.

Install Python 3.12 and create a virtual environment

```
winget install -e --id Python.Python.3.12
# restart VS Code, then:
& "$env:LOCALAPPDATA\Programs\Python\Python312\python.exe" -m venv .venv
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned   # one-time
.venv\Scripts\activate
```

Install dependencies

```
pip install -r requirements.txt
pip install langchain-google-genai
```

Add the API key

Create or edit **.env** in the project folder (the file is git-ignored):

```
GOOGLE_API_KEY=<your key from aistudio.google.com/apikey>
```

Seed the databases

```
python seed_db.py            # business.db: 150 customers, 50 products, 500 orders
python ugly_schema_seed.py   # enterprise.db: 25+ tables with cryptic names
```

Run (two terminals)

```
# Terminal 1 - backend
.venv\Scripts\activate
python api.py          # wait for 'Application startup complete'

# Terminal 2 - UI
.venv\Scripts\activate
python gradio_ui.py    # then open http://localhost:7860
```

Pressing Ctrl + C in a terminal stops the program running there; it does not copy text.

4. Problems hit and how they were fixed

Symptom	Cause	Fix
ModuleNotFoundError: No module named 'dotenv'	Ran agent.py with the VS Code Run button before installing libraries	pip install -r requirements.txt; run the app from the terminal
Failed to build 'zstandard': Microsoft Visual C++ 14.0 or greater is required	Python 3.15 was too new; no prebuilt package available	Installed Python 3.12 and recreated .venv
'py' is not recognized	Python launcher not installed	Called Python 3.12 by its full path to create the venv
running scripts is disabled on this system	Default PowerShell execution policy	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
No Anthropic API credits	A Claude Pro chat plan does not include API credits; the API is billed separately	Switched the LLM to Google Gemini's free tier
AnthropicConnectionError ... Illegal header value b'Bearer '	agent.py still used Claude with an empty key (edits not yet saved)	Saved the Gemini edits in agent.py
503 UNAVAILABLE: model experiencing high demand	Google's gemini-flash-latest was overloaded	Changed model to gemini-flash-lite-latest
UI shows 'Error: timed out'	Free-tier answers took about 60 s; UI waited only 90 s	Raised TIMEOUT in gradio_ui.py from 90 to 300

5. Code changes

agent.py: Claude to Gemini

```
- from langchain_anthropic import ChatAnthropic
+ from langchain_google_genai import ChatGoogleGenerativeAI

- llm = ChatAnthropic(
-     model="claude-haiku-4-5",
-     anthropic_api_key=os.getenv("ANTHROPIC_API_KEY"),
+ llm = ChatGoogleGenerativeAI(
+     model="gemini-flash-lite-latest",
+     google_api_key=os.getenv("GOOGLE_API_KEY"),
      max_tokens=1024,
      temperature=0,
  )

# 4 places: Gemini can return content as a list of parts, so read plain text
- response.content.strip()
+ response.text.strip()
```

gradio_ui.py

```
- TIMEOUT = 90.0
+ TIMEOUT = 300.0
```

The cost figures in the UI still use Claude Haiku prices, so they don't reflect Gemini usage (which is free on this tier).

6. Test runs and outputs

Test 1: direct Python call

Prompt: How many customers are there?

```
python -c "from agent import run_agent; print(run_agent('How many customers are there?'))"
```

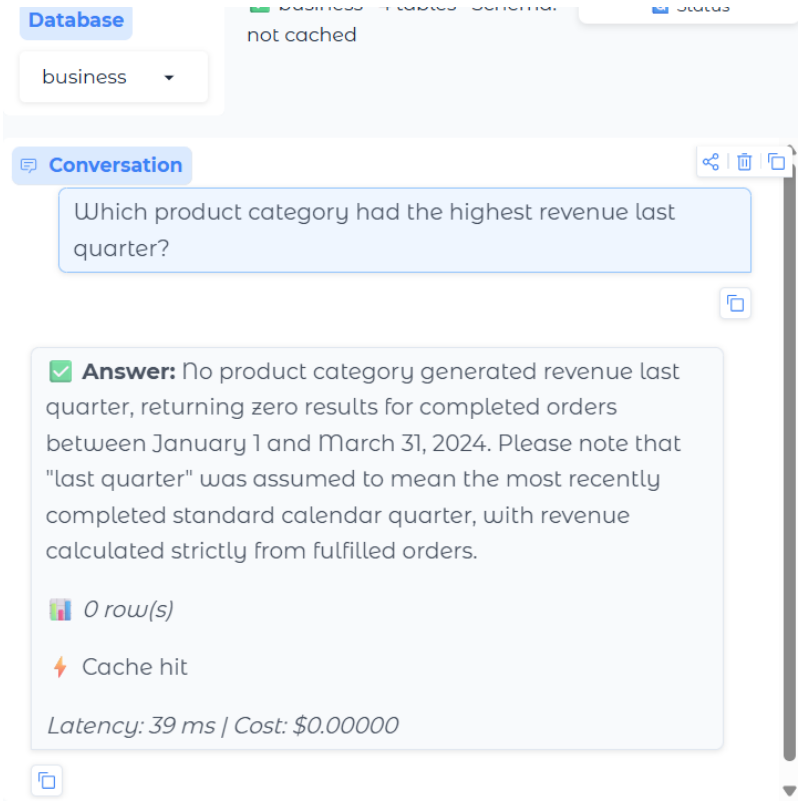
Output (trimmed):

```
sql_query:          SELECT COUNT(*) FROM customers
sql_result:         [{'COUNT(*)': 150}]
summary:            There are a total of 150 customers in the database. Please note
                    that this total count is suspiciously round, which serves as a
                    validation warning.
validation_warning: WARNING: Suspiciously round total count (150).
error:              None
token_usage:        generate_sql 656 in / 5 out, validate 105 / 13, summarize 122 / 31
latency_ms:         60945
```

Result: correct. The seed script creates exactly 150 customers. The 'suspiciously round' warning is a false alarm from the validation step, which is tuned to be cautious.

Test 2: browser UI (Chat tab, business database)

Prompt: Which product category had the highest revenue last quarter?



Output: 0 rows, with the assumption 'last quarter = 1 January to 31 March 2024, completed orders only'.

Result: likely a silent wrong answer. The sample data has orders from January 2023 to June 2024, so Q1 2024 should contain completed orders. The SQL ran without an error but returned nothing, probably because of a filter mismatch such as status = 'Completed' instead of the lowercase 'completed' stored in the data. The lighter Gemini model is more prone to this than Claude Haiku. The 'Cache hit' label shows the answer was reused from an earlier attempt, so the cache also repeats wrong answers until it is flushed (Cache tab, Flush Cache).

Backend log for the session

```
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:50494 - "GET /tables?db_name=business HTTP/1.1" 200 OK
INFO:     127.0.0.1:50496 - "GET /cache/stats HTTP/1.1" 200 OK
INFO:     127.0.0.1:50497 - "GET /health?db_name=business HTTP/1.1" 200 OK
INFO:     127.0.0.1:50495 - "GET /cost/report HTTP/1.1" 200 OK
INFO:     127.0.0.1:50499 - "GET /tables?db_name=business HTTP/1.1" 200 OK
INFO:     127.0.0.1:50502 - "GET /tables/customers?db_name=business&limit=20&o
ffset=0 HTTP/1.1" 200 OK
INFO:     127.0.0.1:50503 - "POST /query HTTP/1.1" 500 Internal Server Error
```

The 500 error on POST /query came from the 503 'high demand' response before switching to the lite model.

7. Takeaways

The pipeline works end to end on a free LLM: browser to FastAPI to LangGraph to Gemini to SQLite and back.

Silent wrong answers are the real risk. Test 2 returned a clean-looking answer with no error. Checking the generated SQL, flushing the cache and running eval_harness.py are how to catch these.

Model choice matters. The free lite model is slower (about 60 s per question here) and less accurate on filters than the paid Claude Haiku the project was designed for.

Keep keys out of git. .env is listed in .gitignore; never commit it or paste the key anywhere public.

Credits: original project by sweta2503, MIT License. Modified here to run on Google Gemini.